# Artificial Intelligence for Digital Characters
# Lecture Summary

Study summary based on ETH Zürich course materials (SS 2026)

July 3, 2026

# Abstract

This document is a lecture-summary style script that closely follows the slides of the course *Artificial Intelligence for Digital Characters* at ETH Zürich. The contribution is editing, refactoring, and additional explanatory prose to make the material easier to study for the written exam.

The summary is organized in numbered chapters that mirror the lecture sequence (L01–L14). Each section explains *why* a technique exists, how it fits into the conversational digital character pipeline, and what trade-offs matter for exams. Where the slides present formulas or algorithms, this document walks through them step by step.

**Disclaimer:** I do not guarantee correctness or completeness, nor is this document endorsed by the lecturers. It is a student study aid derived from publicly available course slides, exercises, and past exams. For authoritative material, refer to Moodle and the official lecture slides.

# Contents

# Chapter 1

# Introduction

## 1.1 Course overview

**Goal:** Understand what a conversational digital character is and which AI techniques are required to build one end to end.

A *digital character* in this course is not merely a 3D avatar or a chatbot in isolation. It is an integrated system that can *listen*, *understand*, *respond*, *speak*, and *move* in a way that conveys personality and emotional state. The central engineering challenge is therefore not any single model, but the *pipeline* that connects perception, language, speech, and animation.

The course covers the following capabilities:

- **Speech recognition** — converting the user's voice into text.
- **Language modeling / chatbots** — generating an appropriate textual response.
- **Speech synthesis** — turning the response text into natural-sounding speech.
- **Animation synthesis** — driving the character's face and body from speech and affect.
- **Affective computing** — detecting and modeling user and character emotion.
- **Autonomous agents** — decision-making beyond fixed scripts, including knowledge graphs and reinforcement learning.

### 1.1.1 Assessment

The final grade is determined by a **written exam** (120 minutes, 120 points). No summary sheet is allowed. A non-programmable calculator and a neutral dictionary (mother tongue to English) are permitted. There is no formal requirement to complete the exercises in order to sit the exam, but the five exercises are closely aligned with exam question types.

Optionally, quizzes in seven lectures can improve the grade: 25 out of 35 correct answers yield a bonus of +0.25 grade points.

## 1.2 The conversational digital character pipeline

At the highest level, interaction follows a loop:

1. The user speaks. A microphone captures audio.
2. **Speech recognition** produces a transcript.
3. Parallel **affective sensing** may analyze video, speech prosody, or biosignals to estimate user emotion, intention, and personality-relevant cues.
4. A **chatbot** (typically a large language model, optionally grounded in a knowledge graph or dialogue policy) generates response text conditioned on the transcript and user state.
5. **Speech synthesis** converts the response text into an audio waveform.

6. **Animation synthesis** drives facial expression, gaze, gesture, and body motion consistent with the speech and the character's intended affect.
7. The character's own emotional state may be updated and fed back into later turns.

This loop explains why the course is structured as a sequence of specialized lectures rather than a single "AI" topic: each stage has different data representations, models, evaluation metrics, and failure modes.

### 1.2.1 Dialogue trees versus language models

Historically, conversational systems used **dialogue trees** or finite state machines: manually authored nodes and branches. Such systems are deterministic, easy to control, and low risk for narrow domains. However, they scale poorly to open-ended conversation and cannot gracefully handle unexpected user input.

Modern systems increasingly rely on **large language models** (LLMs) for open-domain response generation. LLMs trade control for flexibility. The course therefore spends substantial time on both *generation* and *adaptation* techniques (prompt engineering, fine-tuning, retrieval-augmented generation) as well as on classical components that remain essential (speech front-ends, animation, knowledge grounding).

## 1.3 Course map

The fourteen lectures group naturally into three parts:

**Part I — Perception and foundations (L01–L04).** Introduction, affective computing, deep learning preliminaries, and speech recognition. These lectures establish how we sense the user and extract features from raw signals.

**Part II — Language and speech (L05–L09).** Three chatbot lectures plus two speech synthesis lectures. Here the focus shifts to text generation, grounding, multimodality, and spoken output.

**Part III — Embodiment and agency (L10–L14).** Animation (classical and learning-based), autonomous agents with knowledge graphs, real-world applications and ethics, and reinforcement learning for behavior synthesis.

## 1.4 Use cases and motivation

Digital characters appear in education, healthcare, museums, entertainment, and location-based experiences. A recurring case study in the course is **Digital Einstein**: an interactive platform that lets users converse with an Albert Einstein avatar. That project illustrates all pipeline stages: speech input, LLM responses grounded in historical material, neural speech synthesis, and image or video-based presentation.

Other motivating scenarios include:

- **Healthcare:** virtual doctors or psychotherapists that must sound empathetic and trustworthy.
- **Education:** tutors that adapt to student affect.
- **Metaverse / companions:** persistent characters with personality.
- **Museums:** guides that explain exhibits in natural language.

Across these settings, the same technical components reappear; what changes is the domain knowledge, safety requirements, and evaluation methodology.

## 1.5 Historical note on speech synthesis

Speech synthesis has evolved through three broad eras mentioned in the introduction lecture:

1. **Concatenative** methods that stitch recorded speech units.
2. **Statistical parametric** methods with signal-processing vocoders.
3. **Neural end-to-end** systems that predict spectrograms or waveforms directly.

Understanding this progression helps when exams ask you to compare Tacotron, FastSpeech, and classical unit selection: the trade-off is usually between *naturalness*, *controllability*, and *inference speed*.

## 1.6 Exam orientation

Past exams (2024 and 2025) follow a stable structure: a scored multiple-choice block (wrong answers penalized), followed by eight to nine open questions spanning affective computing, attention calculations, Mel filter banks, chatbots, speech synthesis, knowledge graphs, reinforcement learning, and animation (often including a Jacobian IK step). The MC section heavily reuses concepts also tested in the five exercises.

When studying this summary, pay special attention to:

- **False/true traps** in terminology (e.g. Mel scale is logarithmic; dialogue trees are *not* flexible).
- **Calculations** with shown formulas (attention, Mel filters, TransE, IK).
- **Comparisons** between paired techniques (Tacotron vs. FastSpeech, early vs. late fusion, FK vs. IK).

# Chapter 2

# Affective Computing

**Goal:** Detect and model human affect from video, audio, text, and biosignals so that digital characters can respond empathetically.

## 2.1 What is affect?

Human behavior is commonly described as arising from three ingredients: **cognition** (thinking), **affect** (feeling), and **behavior** (acting). *Affect* is broader than emotion alone. It includes moods, attitudes, preferences, and interpersonal stances (e.g. warm vs. distant).

For digital characters, affect matters because users do not judge conversations on factual correctness alone. Prosody, facial expression, and timing strongly influence whether a character feels trustworthy, empathetic, or uncanny. The lecture therefore covers both *theories* that explain emotion and *sensors/algorithms* that estimate affect from data.

Applications highlighted in the slides include education (predicting student affect to improve learning gain), healthcare (depression prevalence estimation, therapeutic agents), and the Digital Einstein pipeline (user emotion in, animated expression out).

## 2.2 Theories of emotion

Emotion theories answer: *in what order do subjective experience, bodily changes, and behavior occur?*

### 2.2.1 James–Lange theory

The **James–Lange theory** claims that emotions arise *after* physiological responses to a stimulus:

$$\text{Stimulus} \rightarrow \text{Physiological response} \rightarrow \text{Emotion}$$

Example: seeing a snake increases heart rate first; the feeling of fear is the interpretation of that bodily state. This is **sequential**, not simultaneous.

**Problem:** Exams often state that James–Lange triggers emotion and bodily changes at the same time. That description matches Cannon–Bard, not James–Lange.

### 2.2.2 Cannon–Bard theory

The **Cannon–Bard theory** proposes that physiological arousal and subjective emotional experience occur **in parallel**, both triggered by the same stimulus via thalamic signals to cortex and autonomic nervous system.

### 2.2.3 Schachter–Singer (two-factor) theory

The **Schachter–Singer theory** adds cognitive interpretation: the same physiological arousal can yield different emotions depending on context. Emotion = arousal + label assigned to the situation.

### 2.2.4 Dimensional models

*Dimensional theories* reduce affect to continuous axes, most commonly:

- **Valence** — pleasant vs. unpleasant.
- **Arousal** — calm vs. excited.
- **Dominance** — in control vs. overwhelmed.

Russell's **core affect** framework places emotions in valence–arousal space (e.g. boredom: low valence, low arousal; engagement: positive valence, high arousal). Exams may ask you to locate a phrase such as "anxious, tense, and helpless" in this space: typically negative valence, high arousal, low dominance.

### 2.2.5 Basic emotions

*Basic emotion* theories (Ekman, Plutchik) posit a small set of innate categories—joy, sadness, disgust, surprise, anger, fear—with characteristic facial expressions and action tendencies (e.g. fight/flight). The Facial Action Coding System (FACS) operationalizes this at the muscle level.

## 2.3 Emotion recognition modalities

Signals can be **non-invasive** (video, speech, text, surface biosensors) or **invasive** (blood hormone levels, neurotransmitter measurement). Facial expression analysis and ECG on the skin surface are non-invasive; blood pressure cuff measurement requires physical interaction and is treated as invasive in exercise solutions.

*Multimodal sensing* combines channels because no single modality is reliable alone. The lecture pipeline is:

$$\text{preprocessing} \rightarrow \text{feature extraction} \rightarrow \text{classification (valence/arousal or categories)}$$

## 2.4 Video-based features

### 2.4.1 Action Units (AUs)

*Action units* are minimal visible facial muscle movements defined by FACS. There are 46 AUs; examples include AU1 (inner brow raiser), AU12 (lip corner puller), and AU26 (jaw drop). Basic emotion templates combine AUs, e.g.:

- **Happiness:** AU6 + AU12
- **Surprise:** AU1 + AU2 + AU5 + AU26
- **Fear:** AU1 + AU2 + AU4 + AU5 + AU7 + AU20 + AU26

Tools such as **OpenFace** output 68 facial landmarks, AU intensities in $[0, 5]$, AU presence flags, gaze, and head pose.

### 2.4.2 Eye blinks and gaze

**Eye blinks** are derived from AU45 evaluated per frame. **Gaze** is often discretized into nine regions; gaze away from a stimulus may indicate distraction or avoidance when viewing disturbing images.

### 2.4.3 Mouth Aspect Ratio (MAR)

MAR measures mouth openness from landmarks:

$$\text{MAR} = \frac{|p_2 - p_8| + |p_3 - p_7| + |p_4 - p_6|}{3 \cdot |p_5 - p_1|}$$

It complements AUs for speaking and surprise detection.

### 2.4.4 Fidgeting

*Fidgeting* captures gross body/face movement energy in video:

1. $f_{\text{temp}} = f_{\text{gray}} - b_{\text{gray}}$ (difference from background model)
2. Threshold $|f_{\text{temp}}| > t$ to binary mask
3. Energy $E$ = percentage of surviving pixels
4. Update background $b' = (1 - \alpha)b + \alpha f$

Statistics (mean, std, min, max of $E$ over frames) feed classifiers.

## 2.5 Biosensors

### 2.5.1 Cardiac signals

**ECG** records electrical heart activity from skin electrodes. **PPG** measures reflected light from blood vessels and can even be estimated from webcam video.

*Heart rate variability (HRV)* quantifies variation in beat-to-beat intervals. Time-domain measures include standard deviation of R–R intervals, RMSSD, and pNN50. **Higher HRV** is often associated with relaxed, adaptive autonomic states; **lower HRV** with stress or anxiety. HRV is influenced by age, posture, breathing, and activity.

### 2.5.2 Skin conductance

*Skin conductance* (electrodermal activity) measures sweating-related conductance of the skin and reflects sympathetic arousal. Responses decompose into:

- **Tonic component** — slow baseline level.
- **Phasic component** — fast event-related peaks after stimuli.

Decomposition uses **convex optimization** (e.g. cvxEDA library).

For each phasic peak, exams may ask you to identify:

1. **Amplitude**
2. **Latency** (stimulus to onset)
3. **Rise time** (onset to peak)
4. **Half-recovery time** (peak to 50% recovery)

## 2.6 Artifact detection in R–R intervals

Physiological streams contain artifacts (movement, missed beats). The lecture presents an algorithm based on quartiles. Given R–R intervals, define:

$$\begin{aligned} \text{QD} &= \frac{Q_3 - Q_1}{2} \\ \text{MAD} &= \frac{\text{MedianBeat} - 2.9 \cdot \text{QD}}{3} \\ \text{MED} &= 3.32 \cdot \text{QD} \\ \text{CBD} &= \frac{\text{MAD} + \text{MED}}{2} \end{aligned}$$

If $|RR_i - RR_{\text{last\_valid}}| > \text{CBD}$, the beat is flagged as an artifact (subject to absolute bounds, e.g. 300–2000 ms).

**Worked example (exam style).** For intervals $\{795, 800, 400, 810, 800, 805\}$ ms, a 400 ms interval implies heart rate $60000/400 = 150$ bpm, which is suspiciously high. With $Q_1 = 795$, $Q_3 = 805$, median = 800, one obtains CBD $\approx 139$ ms. Since $|400 - 800| = 400 >$ CBD, the 400 ms beat is an artifact.

## 2.7 Ground truth and classification

Affective labels often come from the **Self-Assessment Manikin (SAM)** for valence, arousal, and dominance, or from basic emotion/stress questionnaires. Classifiers mentioned include Random Forest, SVM, temporal convolutional networks, and transformers on face images.

The biosensor pipeline therefore ends with supervised learning mapping engineered features to affect dimensions or categories. Video and biosensor channels can be fused at feature level (early fusion) or prediction level (late fusion); chatbot lectures return to this theme in a multimodal setting.

## 2.8 Integration with digital characters

In the full character loop, estimated user affect modulates chatbot prompts, speech prosody, and animation parameters. Conversely, the character's displayed affect must be *synthesized* coherently across face, voice, and text—a theme revisited in speech synthesis (prosody) and animation lectures.

**Problem:** Common exam traps: confusing action units (facial muscles) with whole-body motion; claiming softmax outputs lie in $[-1, 1]$; stating that facial expression analysis is invasive.

# Chapter 3

# Deep Learning Preliminaries

**Goal:** Provide the neural network foundations needed for speech, language, and animation models used later in the course.

## 3.1 The neural network unit

A single computational *neuron* (unit) performs two steps:

$$z = \mathbf{w}^\top \mathbf{x} + b$$
$$a = \sigma(z)$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ weights, $b$ bias, and $\sigma$ a nonlinear **activation function**. Without nonlinearity, stacking layers would collapse to one linear map.

Common activations:

- **ReLU:** $\max(0, z)$ — default in hidden layers; cheap and avoids vanishing gradients for positive inputs.
- **Sigmoid:** maps to $(0, 1)$ — historically common; outputs interpreted as probabilities.
- **Softmax:** normalizes a vector to a probability distribution summing to 1 — used in multi-class classification and language model output layers.

**Training.** A network is trained by minimizing a **loss** (e.g. cross-entropy for classification) via **backpropagation**: gradients flow backward through activations using the chain rule. **Problem:** Exam trap: claiming backprop gradients are independent of activation functions. They are not.

## 3.2 Feedforward networks

A typical **two-layer network** computes hidden activations $\mathbf{h} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b})$ and outputs $\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h})$. Applications in the course include sentiment classification and next-word prediction.

## 3.3 Recurrent Neural Networks (RNNs)

**Goal:** Model sequences by maintaining a hidden state that summarizes past inputs.

Many character-related tasks are sequential: tokens in text, frames in audio/video, motor commands over time. An RNN updates

$$\mathbf{h}_t = \sigma(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$
$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

with shared parameters $(\mathbf{U}, \mathbf{W}, \mathbf{V})$ across time steps. Intuitively, $\mathbf{h}_t$ is a compressed memory of $(\mathbf{x}_1, \ldots, \mathbf{x}_t)$.

### 3.3.1 Uses

- **Language modeling:** predict $P(x_{t+1} \mid x_1, \ldots, x_t)$.
- **Sequence classification:** use final hidden state $\mathbf{h}_T$ (e.g. sentiment).
- **Generation:** autoregressively sample tokens until end-of-sequence.

Training uses **backpropagation through time (BPTT)** by unrolling the network. **Teacher forcing** feeds the ground-truth previous token during training; at inference the model feeds its own predictions, which can cause exposure bias.

### 3.3.2 Vanishing gradients and LSTM

Standard RNNs struggle with **long-range dependencies** because gradients shrink when multiplied through many time steps (**vanishing gradient problem**). **LSTM** modules mitigate this with a **cell state** and gating:

- **Forget gate** — what to erase from memory.
- **Input gate** — what new information to write.
- **Output gate** — what to expose as $\mathbf{h}_t$.

The cell state acts as a highway along which information can flow with less attenuation.

### 3.3.3 Bidirectional and stacked RNNs

A **bidirectional RNN** runs one RNN left-to-right and another right-to-left, concatenating hidden states so each position sees future and past context. This is useful for encoding (e.g. speech frames) but not for autoregressive decoding.

**Stacked RNNs** add depth by feeding hidden states of one layer as inputs to the next.

**Problem:** Exam trap: RNNs process all time steps in parallel like Transformers. They do not; they are sequential.

## 3.4 Encoder–decoder models

Machine translation and dialogue motivated **encoder–decoder** architectures: an encoder RNN (or Transformer) compresses input $(x_1, \ldots, x_T)$ into a context representation; a decoder generates output $(y_1, \ldots, y_m)$ one token at a time:

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_m p(y_m \mid y_1, \ldots, y_{m-1}, \mathbf{x})$$

The decoder is initialized from the encoder's final state (or cross-attends to all encoder states in Transformer models). Encoder and decoder may use different parameters and architectures (LSTM, CNN, Transformer).

## 3.5 Attention mechanism

**Goal:** Allow decoders to focus on relevant parts of the input instead of bottling everything into one fixed vector.

RNN encoder–decoders force all source information through a single context vector $\mathbf{h}_T$, which is painful for long sentences. **Attention** assigns a weight to each source position when producing each target token.

Given query $\mathbf{q} \in \mathbb{R}^d$ and keys/values $(\mathbf{k}_t, \mathbf{v}_t)$:

$$s_t = \frac{\mathbf{q}^\top \mathbf{k}_t}{\sqrt{d}} \quad \text{(scaled dot-product)}$$
$$a_t = \frac{\exp(s_t)}{\sum_{t'} \exp(s_{t'})}$$
$$\mathbf{c} = \sum_t a_t \mathbf{v}_t$$

Scaling by $\sqrt{d}$ prevents dot products from growing too large, which would push softmax into saturated regions with tiny gradients.

### 3.5.1 Self-attention and cross-attention

- **Self-attention:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ all come from the same sequence (encoder or masked decoder).
- **Cross-attention:** queries from decoder attend to encoder keys/values.

**Multi-head attention** runs several attention operations in parallel so the model can capture different relationship types (syntax vs. semantics).

**Exam calculation pattern.** You may be given numeric $\mathbf{q}$ and $\mathbf{x}_t$ and asked for scores, softmax weights, and context vector. Always show: dot products → scale → softmax → weighted sum.

## 3.6 Transformers

Transformers replace recurrence with attention entirely. Because self-attention is **permutation-invariant**, positional information must be injected.

### 3.6.1 Positional encoding

Sinusoidal encodings add to token embeddings:

$$\mathrm{PE}(t, 2i) = \sin\left(t/10000^{2i/d}\right)$$
$$\mathrm{PE}(t, 2i+1) = \cos\left(t/10000^{2i/d}\right)$$

Each position receives a unique pattern across dimensions. Simplified exam versions may use $\mathrm{PE}(t, 0) = \sin(t)$, $\mathrm{PE}(t, 1) = \cos(t)$.

### 3.6.2 Encoder and decoder stacks

Each layer typically contains:

- Multi-head self-attention
- Position-wise feed-forward network
- Residual connections and **layer normalization**

The **decoder** uses **masked** self-attention so position $t$ cannot attend to future positions—at test time those tokens are unknown. Then decoder layers **cross-attend** to encoder outputs.

**Problem:** Exam trap: positional encoding encodes token *content*. It encodes *position* only.

## 3.7 Why this matters for digital characters

Speech recognition (Whisper), chatbots (GPT-style decoders), speech synthesis (Tacotron/FastSpeech attention or FFT blocks), and animation models (Transformers for faces) all inherit these primitives. Later chapters assume you can:

1. Explain why attention beats a single context vector.
2. Compute a small attention example by hand.
3. Contrast RNN sequential processing with Transformer parallelism.
4. Describe LSTM's role against vanishing gradients.

# Chapter 4

# Speech Recognition

**Goal:** Convert spoken audio into text by extracting perceptually motivated features and decoding them with neural acoustic models.

Speech recognition is the entry point of the conversational pipeline: without a reliable transcript, downstream language models cannot run. This lecture connects **phonetics** (what we hear), **signal processing** (how we represent audio), and **sequence models** (how we map features to words).

## 4.1 Phonetics and pitch

*Phones* are speech sounds represented by symbols (ARPAbet or IPA). A word's pronunciation is a **string of phones**. At the waveform level, periodic voiced sounds have a **fundamental frequency** $F_0$ (lowest periodic component). **Pitch** is the perceptual correlate of $F_0$; the relationship is approximately linear below $\sim$1 kHz and more logarithmic above.

The **Mel scale** models perceived pitch spacing:

$$\text{Mel} = 1127 \ln\left(1 + \frac{f}{700}\right), \qquad f = 700\left(e^{\text{Mel}/1127} - 1\right)$$

**Problem:** Exam trap: "The Mel scale is linear in frequency." It is logarithmic/perceptual.

Complex speech sounds sum many sinusoids; a **spectrum** lists frequency components and amplitudes. A **spectrogram** plots frequency content over time and is the standard visual representation in tools like Praat.

## 4.2 Feature extraction pipeline

Speech is non-stationary: frequencies change over time. We therefore analyze **short frames** rather than the entire utterance at once.

### 4.2.1 Windowing and the DFT

Audio is cut into overlapping frames (typically **20–40 ms** windows with $\sim$50% hop). A **Hamming window**

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

tapers frame edges to reduce **spectral leakage** when discontinuities would otherwise spread energy across bins.

Each windowed frame undergoes the **Discrete Fourier Transform (DFT)** (often via FFT):

$$X(k) = \sum_{n=0}^{N-1} x(n)\, e^{-j2\pi kn/N}$$

yielding complex coefficients whose magnitudes form the **power spectrum** $P(k) = |X(k)|^2/N$.

### 4.2.2 Mel filter bank

Human hearing is more discriminative at low frequencies. A **Mel filter bank** applies overlapping **triangular** filters spaced on the Mel scale (not rectangular filters—another exam trap). Construction:

1. Convert lower and upper Hz bounds to Mel.
2. Place $N + 2$ points linearly in Mel for $N$ filters.
3. Convert each Mel point back to Hz.
4. Map Hz to FFT bins; each filter rises from left neighbor to center, then falls to right neighbor.
5. Take the log of filter energies → typically ∼80-dimensional vector per frame.

**Worked example (exercise style).** For 100–2000 Hz with four filters, six Mel endpoints are computed; converting back yields Hz landmarks approximately 100, 320, 600, 959, 1415, 2000 Hz. Filter 1 spans 100–320–600 Hz, filter 2 spans 320–600–959 Hz, and so on. Exam 2024 used 200–8000 Hz with three filters and five Mel points—the method is identical, only numbers change.

### 4.2.3 Augmentation

Training robustness improves with **time warping**, **frequency masking**, and **time masking** of log-Mel features (SpecAugment-style), simulating channel noise and partial occlusions.

## 4.3 Analog-to-digital conversion

Before feature extraction, microphones produce continuous signals that are **sampled** (discrete time) and **quantized** (discrete amplitude levels). Quantization maps real amplitudes to a finite set that can include negative values—it does not force positivity.

## 4.4 Acoustic modeling architectures

Modern recognizers map sequences of feature frames to sequences of characters or word pieces.

### 4.4.1 Encoder–decoder with subsampling

Raw frame rate is high relative to word rate. **Subsampling** (stacking frames, convolutional pooling, or pyramidal RNNs) compresses time before attention or CTC decoding.

### 4.4.2 Connectionist Temporal Classification (CTC)

CTC aligns frame-level outputs with shorter label sequences using a **blank** token. Consecutive identical labels collapse after removing blanks. Many alignments map to the same output string; training maximizes total probability over alignments via dynamic programming. **Independence assumption:** outputs at different times are treated as conditionally independent given audio—simplifying decoding but limiting language modeling unless an external LM is used.

### 4.4.3 RNN Transducer (RNN-T)

RNN-T adds a **prediction network** (language model over previous non-blank outputs) and a **joiner** combining acoustic and linguistic states. Crucially, RNN-T supports **streaming**: when a blank is emitted, advance the acoustic index; when a non-blank symbol is emitted, advance the label index without consuming new frames. **Problem:** Exam trap: "RNN-T is offline only." It supports online/streaming decoding.

### 4.4.4 Whisper

**Whisper** is a Transformer encoder–decoder trained on large-scale weakly supervised multilingual data, performing transcription and related tasks in one model. It represents the end-to-end neural approach that largely supersedes hand-crafted HMM pipelines in research and products.

## 4.5 Word Error Rate (WER)

**Goal:** Evaluate recognizer output against a reference transcript.

$$\mathrm{WER} = 100 \times \frac{S + D + I}{N_{\mathrm{ref}}}$$

where $S$ = substitutions, $D$ = deletions, $I$ = insertions, and $N_{\mathrm{ref}}$ is the number of words in the reference.

**Example.** Reference "portable form of stores last night so" (6 words). System "portable phone upstairs last night so" has two substitutions $\rightarrow$ WER $= 100 \times 2/6 \approx 33.3\%$.

WER is simple and widely used but overweight function words; semantic error rate is desirable but harder to standardize.

To compare two systems, matched-pair statistical tests on sentence segments exist, but WER alone remains the default metric in lectures and exercises.

## 4.6 Prosody connection

Although recognition focuses on lexical content, slides note that **prosody** (pitch, duration, loudness, rhythm) affects both human perception and downstream synthesis. Prosody is *not* purely spectral—exam MC statements claiming otherwise are false.

## 4.7 Role in the digital character pipeline

Speech recognition output feeds the chatbot module; confidence scores and timing can inform turn-taking. Errors propagate: a misrecognized word can derail LLM reasoning, which is why robust front-ends and domain adaptation (custom vocabularies) matter in deployment. The next lectures address how the chatbot generates text and how synthesis reconstructs expressive speech from that text.

**Problem:** Summary of common MC traps for this chapter: Mel filters are triangular; fundamental frequency is the *lowest* prominent frequency, not the highest; WER insertion vs deletion definitions; RNN-T streaming support.